

Notes on Chapter 21 - Databases

The Caveman Coder

2025-08-13

Prerequisites

```
library(DBI)
library(dbplyr)
library(tidyverse)
```

```
## -- Attaching core tidyverse packages ----- tidyverse 2.0.0 --
## v dplyr      1.1.4      v readr      2.1.5
## v forcats    1.0.0      v stringr   1.5.1
## v ggplot2    3.5.2      v tibble    3.3.0
## v lubridate  1.9.4      v tidyr     1.3.1
## v purrr      1.1.0
## -- Conflicts ----- tidyverse_conflicts() --
## x dplyr::filter() masks stats::filter()
## x dplyr::ident()  masks dbplyr::ident()
## x dplyr::lag()    masks stats::lag()
## x dplyr::sql()    masks dbplyr::sql()
## i Use the conflicted package (<http://conflicted.r-lib.org/>) to force all conflicts to become errors
```

Connecting to duckdb:

```
con <- DBI::dbConnect(duckdb::duckdb())
```

Loading data

```
dbWriteTable(con, "mpg", ggplot2::mpg)
dbWriteTable(con, "diamonds", ggplot2::diamonds)
```

DBI basics

Listing all tables in the database using `dbListTables()`:

```
dbListTables(con)
```

```
## [1] "diamonds" "mpg"
```

Reading tables using `dbReadTable()`:

```
con |>
  dbReadTable("diamonds") |>
  as_tibble()
```

```
## # A tibble: 53,940 x 10
##   carat cut      color clarity depth table price      x      y      z
##   <dbl> <fct>    <fct> <fct>    <dbl> <dbl> <int> <dbl> <dbl> <dbl>
```

```
## 1 0.23 Ideal      E      SI2      61.5    55    326    3.95    3.98    2.43
## 2 0.21 Premium    E      SI1      59.8    61    326    3.89    3.84    2.31
## 3 0.23 Good       E      VS1      56.9    65    327    4.05    4.07    2.31
## 4 0.29 Premium    I      VS2      62.4    58    334    4.2     4.23    2.63
## 5 0.31 Good       J      SI2      63.3    58    335    4.34    4.35    2.75
## 6 0.24 Very Good J      VVS2     62.8    57    336    3.94    3.96    2.48
## 7 0.24 Very Good I      VVS1     62.3    57    336    3.95    3.98    2.47
## 8 0.26 Very Good H      SI1      61.9    55    337    4.07    4.11    2.53
## 9 0.22 Fair       E      VS2      65.1    61    337    3.87    3.78    2.49
## 10 0.23 Very Good H      VS1      59.4    61    338    4       4.05    2.39
## # i 53,930 more rows
```

Running SQL queries on the table using `dbGetQuery()`:

```
sql <- "
  SELECT carat, cut, clarity, color, price
  FROM diamonds
  WHERE price > 15000
"

as_tibble(dbGetQuery(con, sql))
```

```
## # A tibble: 1,655 x 5
##   carat cut      clarity color price
##   <dbl> <fct>    <fct>   <fct> <int>
## 1  1.54 Premium    VS2      E     15002
## 2  1.19 Ideal      VVS1     F     15005
## 3  2.1   Premium    SI1      I     15007
## 4  1.69 Ideal      SI1      D     15011
## 5  1.5   Very Good VVS2     G     15013
## 6  1.73 Very Good VS1      G     15014
## 7  2.02 Premium    SI2      G     15014
## 8  2.05 Very Good SI2      F     15017
## 9  1.5   Very Good VS1      F     15022
## 10 1.82 Very Good SI1      G     15025
## # i 1,645 more rows
```

dbplyr basics

Using `tbl()` to create a database object:

```
diamonds_db <- tbl(con, "diamonds")
diamonds_db
```

```
## # Source:   table<diamonds> [?? x 10]
## # Database: DuckDB v1.3.2 [norman@Linux 6.16.0-5-cachyos:R 4.5.1/:memory:]
##   carat cut      color clarity depth table price      x      y      z
##   <dbl> <fct>    <fct> <fct>   <dbl> <dbl> <int> <dbl> <dbl> <dbl>
## 1  0.23 Ideal      E      SI2      61.5    55    326    3.95    3.98    2.43
## 2  0.21 Premium    E      SI1      59.8    61    326    3.89    3.84    2.31
## 3  0.23 Good       E      VS1      56.9    65    327    4.05    4.07    2.31
## 4  0.29 Premium    I      VS2      62.4    58    334    4.2     4.23    2.63
## 5  0.31 Good       J      SI2      63.3    58    335    4.34    4.35    2.75
## 6  0.24 Very Good J      VVS2     62.8    57    336    3.94    3.96    2.48
## 7  0.24 Very Good I      VVS1     62.3    57    336    3.95    3.98    2.47
## 8  0.26 Very Good H      SI1      61.9    55    337    4.07    4.11    2.53
```

```
## 9 0.22 Fair      E      VS2      65.1      61      337      3.87      3.78      2.49
## 10 0.23 Very Good H      VS1      59.4      61      338      4        4.05      2.39
## # i more rows
```

The database object created uses *lazy evaluation*; it only performs the operation when needed.

For example,

```
big_diamonds_db <- diamonds_db |>
  filter(price > 15000) |>
  select(carat:clarity, price)

big_diamonds_db
```

```
## # Source:      SQL [?? x 5]
## # Database: DuckDB v1.3.2 [norman@Linux 6.16.0-5-cachyos:R 4.5.1/:memory:]
##   carat cut      color clarity price
##   <dbl> <fct>    <fct> <fct> <int>
## 1  1.54 Premium  E      VS2     15002
## 2  1.19 Ideal   F      VVS1    15005
## 3  2.1  Premium  I      SI1     15007
## 4  1.69 Ideal   D      SI1     15011
## 5  1.5  Very Good G      VVS2    15013
## 6  1.73 Very Good G      VS1     15014
## 7  2.02 Premium  G      SI2     15014
## 8  2.05 Very Good F      SI2     15017
## 9  1.5  Very Good F      VS1     15022
## 10 1.82 Very Good G      SI1     15025
## # i more rows
```

It doesn't know the number of rows! Why? Because knowing the total number of rows requires executing the complete query first.

Using the `show_query()` function to see the SQL code used to create the db object:

```
big_diamonds_db |>
  show_query()
```

```
## <SQL>
## SELECT carat, cut, color, clarity, price
## FROM diamonds
## WHERE (price > 15000.0)
```

Using `collect()` to get all the data back from a db object created using SQL query:

```
big_diamonds <- big_diamonds_db |>
  collect()

big_diamonds
```

```
## # A tibble: 1,655 x 5
##   carat cut      color clarity price
##   <dbl> <fct>    <fct> <fct> <int>
## 1  1.54 Premium  E      VS2     15002
## 2  1.19 Ideal   F      VVS1    15005
## 3  2.1  Premium  I      SI1     15007
## 4  1.69 Ideal   D      SI1     15011
## 5  1.5  Very Good G      VVS2    15013
## 6  1.73 Very Good G      VS1     15014
```

```
## 7 2.02 Premium G SI2 15014
## 8 2.05 Very Good F SI2 15017
## 9 1.5 Very Good F VS1 15022
## 10 1.82 Very Good G SI1 15025
## # i 1,645 more rows
```

SQL

In this section, we will learn about the relationship between dplyr commands and SQL queries.

Copying the required data (using the built-in `copy_nycflights13()`) to our database:

```
dbplyr::copy_nycflights13(con)
```

```
## Creating table: airlines
## Creating table: airports
## Creating table: flights
## Creating table: planes
## Creating table: weather
flights <- tbl(con, "flights")
planes <- tbl(con, "planes")
```

SQL basics

```
flights |> show_query()
```

```
## <SQL>
## SELECT *
## FROM flights
```

```
planes |> show_query()
```

```
## <SQL>
## SELECT *
## FROM planes
```

Using `WHERE` and `ORDER BY` to control which rows are included and how they are ordered:

```
flights |>
  filter(dest == "IAH") |>
  arrange(dep_delay) |>
  show_query()
```

```
## <SQL>
## SELECT flights.*
## FROM flights
## WHERE (dest = 'IAH')
## ORDER BY dep_delay
```

Showing how `GROUP BY` converts the query to a summary, causing aggregation to happen:

```
flights |>
  group_by(dest) |>
  summarize(dep_delay = mean(dep_delay, na.rm = TRUE)) |>
  show_query()
```

```
## <SQL>
## SELECT dest, AVG(dep_delay) AS dep_delay
## FROM flights
## GROUP BY dest
```

Important distinction between dplyr verbs and SELECT clauses:

- SQL is not case-sensitive. It is only by convention that we write queries in upper-case.
- Order matters in SQL. The proper order is: SELECT, FROM, WHERE, GROUP BY, ORDER BY.
- SQL evaluates the clauses in this order: FROM, WHERE, GROUP BY, SELECT, and ORDER BY.

SELECT

- Performs the same job as `select()`, `mutate()`, `rename()`, `relocate()`, and `summarize()`.

```
planes |>
  select(tailnum, type, manufacturer, model, year) |>
  show_query()
```

```
## <SQL>
## SELECT tailnum, "type", manufacturer, model, "year"
## FROM planes
```

```
planes |>
  select(tailnum, type, manufacturer, model, year) |>
  rename(year_built = year) |>
  show_query()
```

```
## <SQL>
## SELECT tailnum, "type", manufacturer, model, "year" AS year_built
## FROM planes
```

```
planes |>
  select(tailnum, type, manufacturer, model, year) |>
  relocate(manufacturer, .before = type) |>
  show_query()
```

```
## <SQL>
## SELECT tailnum, manufacturer, "type", model, "year"
## FROM planes
```

In the query generated, “year” and “type” are wrapped in double quotes because these are **reserved words** in duckdb. Wrapping them in " avoids any potential confusion between column/table names and SQL operators.

To be safe, we can wrap every column names in ". Other database systems use backticks instead of quotation marks.

- Example of `mutate()` translated into SELECT in SQL:

```
flights |>
  mutate(
    speed = distance / (air_time / 60)
  ) |>
  show_query()
```

```
## <SQL>
## SELECT flights.*, distance / (air_time / 60.0) AS speed
## FROM flights
```

FROM

This is more useful when we are joining tables.

GROUP BY

The **dplyr** `group_by()` is translated to `GROUP BY` and `summarize()` is translated to `SELECT`:

```
diamonds_db |>
  group_by(cut) |>
  summarize(
    n = n(),
    avg_price = mean(price, na.rm = TRUE)
  ) |>
  show_query()
```

```
## <SQL>
## SELECT cut, COUNT(*) AS n, AVG(price) AS avg_price
## FROM diamonds
## GROUP BY cut
```

WHERE

The **dplyr** function `filter()` is translated to `WHERE`:

```
flights |>
  filter(dest == "IAH" | dest == "HOU") |>
  show_query()
```

```
## <SQL>
## SELECT flights.*
## FROM flights
## WHERE (dest = 'IAH' OR dest = 'HOU')
```

```
flights |>
  filter(arr_delay > 0 & arr_delay < 20) |>
  show_query()
```

```
## <SQL>
## SELECT flights.*
## FROM flights
## WHERE (arr_delay > 0.0 AND arr_delay < 20.0)
```

Notable differences:

- the OR pipe `|` becomes `OR`.
- the `&` symbol becomes `AND`.
- comparison is performed using `=` instead of `==`.
- for literal strings, single quotes (`'`) are used instead of double quotes (`"`) because double quotes are used to avoid conflicts with reserved words.
- SQL uses `NULL` instead of `NA`.
- the `%in%` operator somewhat becomes `IN` in SQL

Examples:

```
flights |>
  filter(!is.na(dep_delay)) |>
  show_query()
```

```
## <SQL>
```

```
## SELECT flights.*
## FROM flights
## WHERE (NOT((dep_delay IS NULL)))
```

```
flights |>
  filter(dest %in% c("IAH", "HOU")) |>
  show_query()
```

```
## <SQL>
## SELECT flights.*
## FROM flights
## WHERE (dest IN ('IAH', 'HOU'))
```

While the expression `WHERE (NOT((dep_delay IS NULL)))` works, it is not very “human-readable” – a better way to express this is: `WHERE "dep_delay" IS NOT NULL`.

In **dplyr**, we can use `filter()` after `summarize()`. This is translated to SQL queries by means of a `HAVING` clause:

```
diamonds_db |>
  group_by(cut) |>
  summarize(n = n()) |>
  filter(n > 100) |>
  show_query()
```

```
## <SQL>
## SELECT cut, COUNT(*) AS n
## FROM diamonds
## GROUP BY cut
## HAVING (COUNT(*) > 100.0)
```

ORDER BY

Ordering rows is handled by `ORDER BY` in SQL:

```
flights |>
  arrange(year, month, day, desc(dep_delay)) |>
  show_query()
```

```
## <SQL>
## SELECT flights.*
## FROM flights
## ORDER BY "year", "month", "day", dep_delay DESC
```

The `desc()` function in **dplyr** is directly inspired by SQL’s `DESC`.

Subqueries

In SQL, a subquery is simply a query used as a data source in the `FROM` clause instead of the usual table.

```
flights |>
  mutate(
    year1 = year + 1,
    year2 = year1 + 1
  ) |>
  show_query()
```

```
## <SQL>
## SELECT q01.*, year1 + 1.0 AS year2
```

```
## FROM (
##   SELECT flights.*, "year" + 1.0 AS year1
##   FROM flights
## ) q01
```

```
flights |>
  mutate(year1 = year + 1) |>
  filter(year1 == 2014) |>
  show_query()
```

```
## <SQL>
## SELECT q01.*
## FROM (
##   SELECT flights.*, "year" + 1.0 AS year1
##   FROM flights
## ) q01
## WHERE (year1 = 2014.0)
```

Note: sometimes, the translation to SQL is not yet optimized and plyr will create a subquery when it's not needed.

JOIN

Joins in **dplyr** and SQL are similar:

```
flights |>
  left_join(planes |> rename(year_built = year), join_by(tailnum)) |>
  show_query()
```

```
## <SQL>
## SELECT
##   flights.*,
##   planes."year" AS year_built,
##   "type",
##   manufacturer,
##   model,
##   engines,
##   seats,
##   speed,
##   engine
## FROM flights
## LEFT JOIN planes
##   ON (flights.tailnum = planes.tailnum)
```

Other verbs

To see the full set of what's available in dbplyr, go to <https://dbplyr.tidiverse.org/reference/>.

Function translations

To help understand how column-wise functions get translated into SQL queries, the following helper functions are helpful:

```
summarize_query <- function(df, ...) {
  df |>
    summarize(...) |>
    show_query()
}
```



```

}

mutate_query <- function(df, ...) {
  df |>
    mutate(..., .keep = "none") |>
    show_query()
}

```

Using the helper functions above to understand how complex operations in dplyr can be done using SQL queries:

```

flights |>
  group_by(year, month, day) |>
  summarize_query(
    mean = mean(arr_delay, na.rm = TRUE),
    median = median(arr_delay, na.rm = TRUE)
  )

```

`summarize()` has grouped output by "year" and "month". You can override using the `groups` argument.

```

## <SQL>
## SELECT
##   "year",
##   "month",
##   "day",
##   AVG(arr_delay) AS mean,
##   MEDIAN(arr_delay) AS median
## FROM flights
## GROUP BY "year", "month", "day"

```

Here is another example using the function `OVER` in the translation:

```

flights |>
  group_by(year, month, day) |>
  mutate_query(
    mean = mean(arr_delay, na.rm = TRUE)
  )

```

```

## <SQL>
## SELECT
##   "year",
##   "month",
##   "day",
##   AVG(arr_delay) OVER (PARTITION BY "year", "month", "day") AS mean
## FROM flights

```

It is done this way in SQL because the `GROUP BY` clause in SQL is used exclusively for summaries — the task of grouping is handed over to `OVER`.

The `OVER` clause can also “look” forward or backwards using `LEAD` and `LAG` clauses:

```

flights |>
  group_by(dest) |>
  arrange(time_hour) |>
  mutate_query(
    lead = lead(arr_delay),
    lag = lag(arr_delay)
  )

```

```

)

## <SQL>
## SELECT
##   dest,
##   LEAD(arr_delay, 1, NULL) OVER (PARTITION BY dest ORDER BY time_hour) AS lead,
##   LAG(arr_delay, 1, NULL) OVER (PARTITION BY dest ORDER BY time_hour) AS lag
## FROM flights
## ORDER BY time_hour

```

Since SQL tables are designed to handle large sets of data, the tables have no intrinsic order. In the **dplyr** code above, if the `arrange()` verb is not used, one might get different results each time the code is run.

The **dplyr** expressions `if_else()` and `case_when()` are translated to `CASE WHEN` in SQL:

```

flights |>
  mutate_query(
    description = if_else(arr_delay > 0, "delayed", "on-time")
  )

```

```

## <SQL>
## SELECT CASE WHEN (arr_delay > 0.0) THEN 'delayed' WHEN NOT (arr_delay > 0.0) THEN 'on-time' END AS d
## FROM flights

```

```

flights |>
  mutate_query(
    description = case_when(
      arr_delay < -5 ~ "early",
      arr_delay < 5 ~ "on-time",
      arr_delay >= 5 ~ "late"
    )
  )

```

```

## <SQL>
## SELECT CASE
## WHEN (arr_delay < -5.0) THEN 'early'
## WHEN (arr_delay < 5.0) THEN 'on-time'
## WHEN (arr_delay >= 5.0) THEN 'late'
## END AS description
## FROM flights

```

Other functions in R that don't have a direct translation to SQL can also be handled by `CASE WHEN`:

```

flights |>
  mutate_query(
    description = cut(
      arr_delay,
      breaks = c(-Inf, -5, 5, Inf),
      labels = c("early", "on-time", "late")
    )
  )

```

```

## <SQL>
## SELECT CASE
## WHEN (arr_delay <= -5.0) THEN 'early'
## WHEN (arr_delay <= 5.0) THEN 'on-time'
## WHEN (arr_delay > 5.0) THEN 'late'
## END AS description

```

```
## FROM flights
```

While **dbplyr** translations are not (yet) perfect, they work well especially on commonly-used functions.